

BALANCE — mapa pokręteł (dla Olo)

Wszystkie liczby balansu siedzą w **plikach configu**: `src/ReplicatedStorage/Content/Config/`.

Serwis/kontroler **nigdy nie hardkoduje** balansu — czyta z configu przez agregator `GameConfig.lua` (`GameConfig.Wave`, `GameConfig.Units`, `GameConfig.SkillEffect`, ...). Zmieniasz liczbę → Rojo synchronizuje do Studio → działa.

Nie ruszaj plików w `Services/` ani `Controllers/` dla balansu — tylko configi niżej.

Po edycji: zapis pliku wystarczy (Rojo live-sync). Nic nie kompilujesz ręcznie.

Numery linii = orientacyjne (mogą się przesunąć po edycji pliku). **Nazwa parametru = źródło prawdy** — szukaj po nazwie.

Ścieżki skrócone: `Config/X.lua` = `src/ReplicatedStorage/Content/Config/X.lua`.

Spis pokręteł wg tematu:

1. Wrogowie — HP / DMG / zasięg / prędkość (per tier + boss)
2. Skalowanie fal w nieskończoność (HP/DMG/monety rosną z falą)
3. **Ilość wrogów w fali** (harmonogram ataku)
4. Bossy, mini-bossy, odblokowywanie tierów
5. Nasze jednostki i bloki — HP / obrona / DMG (DPS)
6. Rdzeń (serduszko bazy)
7. Blokowanie ruchu + strzelanie gracza (bronie FPS)
8. Ekonomia — monety za kille + **zarobek offline**
9. Drabina szczęścia — rzadkość / mutacje (koniczyny) / warianty
10. Efekty skill-tree (gałąź bojowa + roll) — **plik dla Olo**
11. Efekty statowe skilli (luck / dmg / fireRate / rollSpeed / mutChance)
12. Ceny umiejętności
13. Lewelowanie jednostek (coin sink)
14. Indeks — bonus luck za % kolekcji
15. Siatka budowania
16. Timingi teatru rolla (kosmetyka)
17. Bazy tabeli Stats

1. Wrogowie — HP, DMG, zasięg, prędkość (per tier + boss)

Plik: `Config/WaveConfig.lua` → tabela `enemyTypes` (linie 55–81)

Polą każdego wroga:

pole	co robi
<code>hp</code>	HP wroga (ile wytrzyma)
<code>speed</code>	prędkość marszu (pathT/s; 0..1 = start→rdzeń). Większe = szybszy
<code>atkDmg</code>	DMG do NASZYCH jednostek (mitygowane przez <code>def</code> jednostki/bloku)

pole	co robi
<code>atkRange</code>	zasięg broni — do jednostek oraz do rdzenia (w tym dystansie staje i strzela)
<code>atkRate</code>	strzały/s do jednostek (większe = szybciej)
<code>coreDmg</code>	DMG do rdzenia za jeden strzał
<code>coreAttackInterval</code>	co ile sekund strzela w rdzeń
<code>tier</code>	numer tieru (1–6); wizual/dobór, nie ruszaj do prostego strojenia
<code>model</code>	model wizualny z <code>Assets.Units</code> (NIE balans)

Tiery — linie wpisów w `enemyTypes` :

tier	klucz	linia	hp	atkDmg	coreDmg
1	<code>grunt</code>	58	30	6	4
2	<code>t2_soldier</code>	60	55	9	6
2	<code>t2_swat</code>	61	110	8	7
2	<code>t2_sniper</code>	62	45	18	8
3	<code>t3_flame</code>	64	130	6	9
3	<code>t3_rocket</code>	65	95	22	12
4	<code>t4_heavy</code>	67	260	12	13
4	<code>t4_jeep</code>	68	340	14	15
5	<code>t5_tank</code>	70	650	40	20
5	<code>t5_heli</code>	71	430	16	18
6	<code>t6_jet</code>	73	520	30	24

Bossy (`isBoss = true` , osobne wpisy):

klucz	linia	pojawia się	hp (bazowe ¹)	coreDmg	archetyp
<code>boss</code>	78	fala 20	900	30	ŚCIANA (wolny mur HP)
<code>boss5</code>	79	fala 40	1800 (<code>hpMult=0.55</code>)	42	SZARŻA (szybki, mniej HP)
<code>boss6</code>	80	fala 60	3200	82	BOMBARDIER (ostrzał rdzenia z dużego dystansu)

¹ **Uwaga:** realne HP bossa NIE pochodzi wprost z pola `hp` (jest inertne) — liczy je `WaveService` ze wzoru `bossHpVsPrevSoldier × HP najsilniejszego zwykłego wroga z fali (n-1) × (per-boss hpMult albo 1)` . Pole `hpMult` (tylko `boss5=0.55`) koryguje to w dół. `speed` / `atk*` / `coreDmg` bossa są czytane wprost.

2. Skalowanie fal w nieskończoność (HP/DMG/monety rosną z falą)

Plik: `Config/WaveConfig.lua` → tabela `scaling` (linia 24)

pole	co robi	teraz
<code>poolBase</code>	(legacy pula wrogów — patrz §3, liczbę wrogów daje teraz <code>attack</code>)	20
<code>poolPerWave</code>	(legacy, jw.)	4

pole	co robi	teraz
hpGrowth	mnożnik HP wrogów za falę → $hp \times hpGrowth^{(n-1)}$	1.12
dmgGrowth	mnożnik atkDmg wrogów za falę	1.07
coinGrowth	mnożnik nagrody monet za falę	1.15

Prototyp „mniej, grubszych”: `Config/WaveConfig.lua` → `proto` (linia 32)

- `countDiv` (=1) — dzielnik liczby wrogów. **1 = liczba dosłowna z `attack`** (nie dziel).
- `hpMul` (=2) — mnożnik HP każdego wroga (grubsi wrogowie).
- `coreDmgMul` (=1) — mnożnik obrażeń do rdzenia.

`hpGrowth=1.12` rośnie szybko ($\times \sim 9.6$ HP po 20 falach). Jak za mocno — zjedź do 1.06–1.09.

3. Ilość wrogów w fali (harmonogram ataku) ★

Plik: `Config/WaveConfig.lua` → tabela `attack` (linie 40–50)

To jest **żywe pokrętko liczby wrogów** (nie `scaling.pool`). Wrogowie celują w środkowe kolumny bazy; ilość i szerokość celu rosną z falą.

pole	linia	co robi	teraz
counts	41	jawna liczba wrogów dla fal początkowych <code>{[1]=4, [2]=7}</code>	4, 7
baseCount	42	liczba wrogów na fali 3	8
countStep	43	+ile wrogów na każdą kolejną falę (od fali 3)	1
countCap	44	SUFIT liczby wrogów (fala nie przekroczy)	40
colsBaseWaves	45	jawna liczba kolumn celu dla fal początkowych <code>{[1]=1, [2]=2}</code>	1, 2
colsBase	46	liczba kolumn celu od fali 3 (środek {4,5,6})	3
colsHoldUntil	47	do tej fali trzyma <code>colsBase</code> ; potem rozszerza	8
colsStep	48	+ile kolumn na krok rozszerzania	1
colsEvery	49	co ile fal + <code>colsStep</code> kolumny (2 = pełne 9 kolumn dopiero ~fala 19)	2

Prędkości (rozjazd): `speedJitter` (linia 52) `{min=0.75, max=1.30}` — każdy wróg dostaje deterministyczny mnożnik prędkości z tego przedziału (nie idą równo).

Tempo fal: `interWaveDelay` (linia 53) = 5.5 s — przerwa między wyczyszczeniem fali a auto-startem następnej.

Skład fali generowanej wzorem (roj taniego mięsa, elity rzadkie):

- `fodderFrac` (linia 113) = 0.35 — udział puli na „fodder floor” (najtańsze tiery zawsze obecne).
- `fodderTiers` (linia 114) = `{1, 2}` — które tiery to fodder.

4. Bossy, mini-bossy, odblokowywanie tierów

Plik: `Config/WaveConfig.lua`

pole	linia	co robi	teraz
<code>bossHpVsPrevSoldier</code>	54	HP bossa = tyle × HP najsilniejszego zwykłego wroga z fali (n-1)	5
<code>tierBase</code>	91	najwyższy tier na falach 1.. <code>bossWave</code> -1	3
<code>tierWindow</code>	93	ile ostatnich tierów aktywnych w mixie (4 → fala 21 = tier 1–4)	4
<code>bossTiers</code>	94	który boss na której fali bossa <code>{boss, boss5, boss6}</code>	↓
<code>bossWave</code>	95	co ile fal DUŻY boss (20 = fala 20,40,60...)	20
<code>miniBossEvery</code>	96	co ile fal MINI-BOSS (elita najwyższego tieru + pasek HP)	5
<code>miniBossHpMult</code>	97	HP mini-bossa = tyle × najsilniejszy zwykły żołnierz tej fali	2.5
<code>bossRewardMult</code>	108	ile× nagroda monet za falę bossa	3
<code>bossGrunts</code>	109	ilu zwykłych wrogów towarzyszy bossowi	10

Reguła tierów: `maxTier = tierBase + floor(fala / bossWave)` (do 6).

Fale 1–19 → tier 3 · fala 20 boss tier4 · fale 21–39 → tier 4 · fala 40 boss tier5 · itd.

Nazwy bossów (kosmetyka, brainrot/meme): `bossNames` (linie **101–107**) — `ranks` + `names`, podmień dowolnie.

Fala 1 ma jawny skład: `waves[1]` (linia **13**, teraz `grunt ×24`, `coinReward=150`).

5. Nasze jednostki i bloki — HP, obrona, DMG (DPS)

Plik: `Config/UnitsConfig.lua`

Zasada DPS: `base DPS = combat.dmg × combat.fireRate`, monotoniczny z rzadkością (`rarityN`) — rzadsze = mocniejsze.

Lewelowanie (§13) mnoży na wierzchu. Zasięg globalnie × `SimConfig.unitRangeMult` (teraz 1.0).

pole	co robi
<code>hp</code>	HP jednostki (ile wytrzyma ostrzał)
<code>def</code>	OBRONA — obrażenia wroga = <code>max(0, atkDmg - def)</code> (patrz też §7 <code>defMinChipFrac</code>)
<code>combat.dmg</code>	DMG jednostki na strzał
<code>combat.fireRate</code>	strzały/s jednostki
<code>combat.range</code>	zasięg jednostki
<code>rarityN</code>	rzadkość dropu (1inN) — NIE bojowe, ale steruje cost lvl (§13)

Jednostki bojowe (Weapon):

klucz	linia	rarityN	hp	def	dmg	fireRate	DPS
<code>noob_soldier</code>	25	3	80	–	4	1.0	4
<code>soldier</code>	30	8	140	–	8	1.2	9.6
<code>sniper</code>	35	15	120	–	35	0.4	14
<code>swat</code>	40	30	500	6	13	1.4	18.2
<code>rocket_soldier</code>	45	60	200	–	56	0.5	28

klucz	linia	rarityN	hp	def	dmg	fireRate	DPS
flamethrower	50	100	400	–	10	4.0	40
tank_soldier	55	150	900	12	9	6.0	54
jeep	60	300	700	8	54	1.5	81
helicopter	71	800	900	6	30	5.0	150
tank	66	1500	1500	22	430	0.6	258
jet	76	2000	700	–	285	1.2	342

Bloki (ściany, pasywne — HP + def, nie strzelają):

klucz	linia	rarityN	hp	def
test_block_wood	88	5	500	10
test_block_stone	92	15	1200	22
test_block_iron	96	40	3000	45
test_block_rusty	100	150	6000	80

Bronie testowe (Weapon): test_weapon_pistol/rifle/cannon/launcher (linie 105–124) — combat per wpis.

6. Rdzeń (serduszko bazy)

- Bazowe HP rdzenia:** Config/StatBaseConfig.lua → coreHP (linia 19, teraz 200).

Design: nieskończone przetrwanie, gracz w końcu zawsze przegrywa (monetyzacja).

- Zasięg ostrzału rdzenia** = atkRange danego wroga (\$1). Wróg staje w tym dystansie i strzela w rdzeń;

coreDmg + coreAttackInterval = jego obrażenia/tempo na rdzeń.

- HP rdzenia to obecnie STAŁA 200** — w drzewku NIE ma jeszcze węzła podnoszącego HP rdzenia

(komentarz core_1 +25/rank w configu to plan, nie implementacja). Chcesz twardszy rdzeń → zmień coreHP.

7. Blokowanie ruchu + strzelanie gracza (bronie FPS)

Plik: Config/SimConfig.lua

pole	linia	co robi	teraz
simHz	5	tick symulacji/s (KNOB wydajności)	10
spawnStagger	6	odstęp pathT między wrogami przy spawnie	0.03
blockStopDist	8	dystans środek-środek, przy którym wróg staje przed placementem	6.5
blockLaneHalf	9	max boczne odchylenie, by placement liczył się jako przeszkoda	3.6
blockAhead / blockRadius	10– 11	legacy (nieużywane po zmianie na standoff)	2.5 / 4.0
playerShotDmg	12	fallback dmg/strzał broni FPS gracza (brak wpisu per-bron)	10

pole	linia	co robi	teraz
playerWeaponDmg	13	dmg/strzał per BROŃ {Pistol=10, AR=8, Shotgun=24, Sniper=60}	↓
headshotMult	14	mnożnik obrażeń za HEAD SHOT (1.0 = tylko wizual)	1.5
unitRangeMult	15	globalny mnożnik zasięgu WSZYSTKICH jednostek gracza	1.0
defMinChipFrac	16	DEF blokuje max (1-frac)=80% obrażeń; wróg ZAWSZE chipuje ≥ frac × atkDmg (koniec deadlocka)	0.2

8. Ekonomia — monety za kille + zarobek offline

Plik: Config/EconomyConfig.lua

pole	linia	co robi	teraz
killReward	5	monety za zabicie wroga (baza; × Stats.coinMult)	2
offline.capHours	4	max godzin naliczania offline	10
offline.ratePerHourFromStats	4	monety/h = Stats.offlineRate (z \$17) gdy true	true
offline.rollsPerHour	4	rolki/h offline (placeholder)	20

Stawka monet/h offline = Config/StatBaseConfig.lua → offlineRate (linia 21, teraz 10), modyfikowana skillami (\$10 offline.mult).

Mnożnik monet (za kille): StatBaseConfig.coinMult (linia 20, teraz 1.0).

9. Drabina szczęścia — rzadkość / mutacje (koniczyny) / warianty

Rzadkość jednostki (tier revealu z rarityN): Config/RarityConfig.lua

- tiers (linie 5–11): progi maxN → nazwa + revealTier (Common ≤10, Uncommon ≤50, Rare ≤200, Epic ≤1000, Legendary ∞).
- luck.base (linia 12) = 1.0.
- pity (linia 13): threshold=100 rolek bez N≥guaranteeMinN(60) → wymuś tier Rare+.

Mutacje (koniczyny x2..x128): Config/MutationConfig.lua

- ladder (linie 9–17): kaskada wspinająca; każdy szczebel ma mult + step (szansa wejścia, × Stats.mutChance).

Linie: x2 = 10, x4 = 11, x8 = 12, x16 = 13, x32 = 14, x64 = 15, x128 = 16.

- baseCloverTier (linia 20) = 3 → domyślnie koniczyny max do x8.
- tierUnlock (linia 21): węzeł drzewa → podnosi sufit (n53 = x16, n63 = x32, n64 = x64).

Warianty (regular/shiny/galaxy — wizual + kolekcja): Config/VariantConfig.lua

- ladder (linie 11–14): galaxy chanceN=1000 (linia 12), shiny chanceN=50 (linia 13). Skala × Stats.variantLuck.

10. Efekty skill-tree (gałąź bojowa + roll) ★ PLIK DLA OLO

Plik: `Config/SkillEffectConfig.lua` — TYLKO parametry efektu (dmg/promień/szansa/cooldown). Ceny osobno (\$12).

efekt	linie	pola	teraz
Explosive Ammo	9–14	<code>baseChance</code> (0.10), <code>tierMult</code> (n78=1..n98=5), <code>splashRadius</code> (9), <code>splashFrac</code> (0.6)	↓
Strike'i	19–23	<code>convoy</code> (n106: cd22, dmg60, r14), <code>airRaid</code> (n108: cd32, dmg180, r16), <code>nuke</code> (n107: cd95, dmg900, r42)	↓
Bonus tier-up	26–28	<code>tierUpChance</code> (n39..n46, suma do 60%)	↓
Offline mult	31–34	<code>mult</code> (n22..n30, suma do +150%), <code>unlockNode</code> (n3)	↓
Gold Roll	39	<code>mult=3</code> , <code>everyByNode</code> (n4=12..n10=6 rolek/proc)	↓
Diamond Roll	40	<code>mult=8</code> , <code>everyByNode</code> (n8=30..n14=18)	↓
Rainbow Roll	41	<code>mult=40</code> , <code>everyByNode</code> (n15=100..n104=30)	↓
Friends' Luck	46–48	<code>pctByNode</code> (+luck za każdego znajomego na serwerze; n74=0.05..n103=0.35)	↓
Double Roll	52–54	<code>chanceByNode</code> (szansa 2. rolki naraz; n54=0.05..n68=0.65)	↓
Automatic Roll	60	<code>regenPerSec</code> (0.7), <code>cost</code> (1), <code>interval</code> (8s), <code>perCycle</code> (2 reele)	↓

11. Efekty statowe skilli (luck / dmg / fireRate / rollSpeed / mutChance)

Plik: `Config/SkillConfig.lua` — tu drzewo mapuje węzły na płaskie Staty. (Ceny z `SkillTreeData` / `SkillCosts`.)

grupa	linie	co robi
<code>LUCK_MULT</code>	57–63	mnożnik roll-luck węzłów Luck I..God Luck III (n70=1.5 .. n90=70)
<code>COMBAT_EFFECT</code>	73–78	<code>n105</code> =dmgMult +10%, <code>n109/110/111</code> =fireRateMult (+5/+5/+20%)
<code>MORE_EFFECT</code>	88–97	Fast Rolls → <code>rollSpeed</code> (n31..n37, razem -1.0: 4.0→3.0); Clover → <code>mutChance</code> (n32,n51,n52,n57..n62)
<code>costExp</code>	108	wykładnik kosztu (maxRank=1 → cost=base)

Struktura drzewa (rodzice, etykiety, domyślne koszty) = `Config/SkillTreeData.lua` — nie ruszać dla balansu liczb.

12. Ceny umiejętności

Plik: `Config/SkillCosts.lua` — TYLKO TU zmieniasz ceny (nadpisuje `SkillTreeData`).

- Każdy wpis: `nX = { coin = ..., roll = ... }`. Wpisz jedną walutę (>0), drugą zostaw 0.
- Obecne wpisy (linie **14–20**): gałąź bojowa `n105–n111` (250k .. 5M coin).
- Brak wpisu dla węzła = domyślny koszt z `SkillTreeData`. Chcesz przecenić stary węzeł? Dopisz linijkę (np. `n70 = { coin = 200, roll = 0 }`).

13. Lewelowanie jednostek (coin sink)

Plik: `Config/LevelConfig.luau` — poziom per typ jednostki; efekt multiplikatywny (`stat = base × growthPerLevel^level`).

pole	linia	co robi	teraz
<code>growthPerLevel</code>	9	mnożnik efektu na poziom (weapon→dmg, block→hp)	1.10
<code>costGrowth</code>	10	wzrost kosztu kolejnego poziomu (<code>baseCost × costGrowth^level</code>)	1.5
<code>baseCostK</code>	11	<code>baseCost = floor(rarityN × baseCostK)</code> — rzadsze drożej	5
<code>maxLevel</code>	12	twardy sufit poziomu	100

14. Indeks — bonus luck za % kolekcji

Plik: `Config/IndexConfig.luau`

- `categories` (linie **12–17**): 4 zakładki (regular/shiny/galaxy/enemies).
- `luckThresholds` (linie **18–22**): próg %ukończenia → bonus luck (25%→+0.5, 50%→+1.0, 100%→+2.0). Semantyka: najwyższy osiągnięty próg.

15. Siatka budowania

Plik: `Config/GridConfig.luau`

- `width` (9) / `depth` (12) / `cellSize` (6, == fizyczny Tile — **nie ruszać**) / `baseCellBudget` (linia **9**, =40) — budżet zajętych kratek (suma footprintów). To realny limit „ile postawisz”.

16. Timingi teatru rolla (kosmetyka, nie twardy balans)

Plik: `Config/RevealConfig.luau` → `tiers` (linie **5–9**): per `revealTier` `buildupSec` / `holdSec` / `cascade` / `cinematic`. Klient interpoluje między zdefiniowanymi tierami.

Auto-roll / double-roll reveal timingi (2.5 s reveal, 2.5 s hold) są w PACK-u `ReplicatedStorage.Roll` (place-only, nie w git) — to strefa dewelopera, nie olo.

17. Bazy tabeli Stats (punkty startowe gracza)

Plik: `Config/StatBaseConfig.luau` (linie **11–23**) — od tych baz `StatProfileService` startuje i dokłada delty skilli/boostów.

pole	linia	co robi	teraz
luck	12	baza szczęścia (z RarityConfig)	1.0
rollSpeed	13	cooldown rolki [s] start (Fast Rolls schodzi do floor)	4.0
mutChance	14	mnożnik szansy mutacji (koniczyn)	1.0
cellBudget	15	budżet kratek (z GridConfig)	40
dmgMult	16	mnożnik dmg wież	1.0
fireRateMult	17	mnożnik szybkostrzelności	1.0
rangeMult	18	mnożnik zasięgu	1.0
coreHP	19	HP rdzenia (§6)	200
coinMult	20	mnożnik monet (za kille)	1.0
offlineRate	21	monety/h offline (§8)	10
variantLuck	22	mnożnik szansy wariantu (shiny/galaxy)	1

Ściąga „chcę zmienić X”:

- **HP wroga / tiera** → `WaveConfig.enemyTypes[typ].hp` · **HP bossa** → `bossHpVsPrevSoldier` (+ per-boss `hpMult`)
- **DMG wroga do jednostek** → `atkDmg` · **do rdzenia** → `coreDmg`
- **Ile wrogów w fali** → `WaveConfig.attack` (`baseCount` , `countStep` , `countCap`)
- **Jak szybko rośnie trudność** → `scaling.hpGrowth` / `dmgGrowth` (+ `proto.hpMul`)
- **HP naszych** → `UnitsConfig[jed].hp` · **obrona** → `def` · **DMG/DPS** → `combat.dmg × fireRate`
- **HP rdzenia** → `StatBaseConfig.coreHP` · **Co ile boss** → `WaveConfig.bossWave`
- **Zarobek offline** → `StatBaseConfig.offlineRate` + `EconomyConfig.offline` + `SkilleffectConfig.offline.mult`
- **Szansa koniczyn** → `MutationConfig.ladder[*].step` · **sufit koniczyn** → `baseCloverTier` / `tierUnlock`
- **Ceny skilli** → `SkillCosts.luau` · **Efekty skilli bojowych/rolla** → `SkilleffectConfig.luau`

Do dopisania / potwierdzenia (rozwijane przez sesję)

- ✓ **Robux / gamepassy / dev products** — `Services/PurchaseService.luau` to STUB (`Init` / `Start` puste).
Brak monetyzacji Robux do balansu na teraz — dojdzie później.
- ✓ **core-HP node** — potwierdzone: w `SkillTreeData` NIE ma węzła HP rdzenia; `coreHP` to płaska stała 200 (§6).
- ✓ `scaling.poolBase/poolPerWave` — potwierdzone LEGACY: liczbę wrogów daje `attack` (WaveService:192 `waveCount`). `hpGrowth/dmgGrowth/coinGrowth` są ŻYWE (CombatService/EconomyService/WaveService).
- **Boosty czasowe** (jeśli dojdą) — mnożniki + czas trwania.
- **Auto Wave** (skill n5) — sprawdzić, czy ma własne parametry tempa (kontroler klienta).
- **Auto Roll** — parametry `SkilleffectConfig.autoRoll` udokumentowane (§10); potwierdzić realną konsumpcję rolek vs regen przy balansie ekonomii.